DocInsight AI: An Intelligent Document Analysis and Question Answering System

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

DSA0306 - NATURAL LANGUAGE PROCESSING

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

Vasantha Nithi D Y (192424137)

Under the Supervision of

Dr. Kumaragurubaran T

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "DocInsight AI: An Intelligent Document Analysis and Question Answering System" has been carried out by Vasantha Nithi D Y 192424137 under the supervision of Dr Kumaragurubaran T and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence and Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr Devi T
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr Kumaragurubaran T
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I Vasantha Nithi D Y 192424137 of the B.Tech Artificial Intelligence and Data Science SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "DocInsight AI: An Intelligent Document Analysis and Question Answering System is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 07-08-2026

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
G Aswartha Manidheep – 192425089	Module 1 – Intelligent Document Processing	Document upload, PDF/DOCX extraction, OCR and preprocessing	Tested document extraction and OCR with different file formats	introduction, Problem Statement, Module 1	25 %
Mohammed Jameel A – 192425057	Module 2 – Document Intelligence & Insight Generation	NLP processing, summarization, keyword/key-phrase extraction and NER	Tested summaries, keywords and entity extraction	Objectives, Literature Review, Module 2	25%
Vasantha Nithi D Y – 192424137	Module 3 – Semantic Search & Knowledge Retrieval	Document chunking, sentence embeddings, vector database and semantic search	Tested similarity search and relevance of retrieved chunks	Existing System, Proposed System, Module 3	25%
Aswanth N N – 192424123	Module 4 – AI-Powered Q&A & Language Assistance	RAG pipeline, LLM integration, Q&A, translation and text simplification	Tested question answering, retrieved context and generated responses	Implementation, Results, Future Work, Module 4	25%
Total					100%

ABSTRACT

In the modern digital environment, large amounts of information are stored and shared through documents such as research papers, academic notes, reports, manuals, business documents, policies, and technical files. Although digital documents provide convenient storage and distribution, extracting specific information from lengthy documents can be time-consuming and difficult. Traditional document viewers mainly provide keyword-based search, which may fail when the wording of a user's query differs from the terminology used in the document. Scanned and image-based documents create an additional challenge because their content may not be directly searchable.

The proposed DocInsight AI: An Intelligent Document Analysis and Question Answering System addresses these limitations by providing an intelligent platform for interacting with uploaded documents. The system accepts supported document formats, extracts textual information, and processes the content using document processing and Optical Character Recognition (OCR) techniques. The extracted text is cleaned, organized, and divided into meaningful sections for further analysis.

The system applies Natural Language Processing (NLP) techniques to generate document insights such as summaries, keywords, key phrases, and important entities. For intelligent information retrieval, document sections are converted into semantic embeddings and stored in a vector database. When a user submits a natural-language question, the question is converted into a semantic representation and compared with the stored document representations. The most relevant sections are retrieved and supplied as context to a Large Language Model through a Retrieval-Augmented Generation (RAG) approach.

The proposed system aims to provide context-aware answers based on the uploaded document while reducing the time and effort required for manual document analysis. Additional capabilities such as translation and text simplification can improve accessibility and understanding of complex content. The system can be useful for students, researchers, professionals, and organizations that regularly work with large volumes of textual information.

Keywords: Document Intelligence, Natural Language Processing, Semantic Search, OCR, Retrieval-Augmented Generation, Question Answering

TABLE OF CONTENTS

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
AI	Artificial Intelligence
NLP	Natural Language Processing
OCR	Optical Character Recognition
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
API	Application Programming Interface
PDF	Portable Document Format
DOCX	Microsoft Word Open XML Document
FAISS	Facebook AI Similarity Search

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The rapid growth of digital technology has resulted in the widespread use of electronic documents across education, research, business, healthcare, government, and technical fields. Information is increasingly stored in formats such as PDF files, Word documents, scanned images, research papers, reports, technical manuals, and organizational records. While digital storage has made information easier to preserve and distribute, extracting useful knowledge from large volumes of documents remains a significant challenge.

Users often need to spend considerable time reading lengthy documents to locate particular information. Traditional document viewers generally provide features such as document viewing and keyword-based searching. However, keyword search depends heavily on the exact words entered by the user and may not understand the meaning or intent behind a query.

For example, a user may ask, “What is the purpose of this study?”, while the document may describe the same information using terms such as aim, objective, or goal. A traditional keyword-based system may not effectively recognize this relationship.

Recent developments in Artificial Intelligence, Natural Language Processing, semantic embeddings, vector databases, Optical Character Recognition, and Large Language Models provide opportunities to overcome these limitations. OCR can convert text contained in scanned documents into machine-readable content, while semantic search can identify information based on meaning rather than exact keyword matching.

DocInsight AI combines these technologies to transform static documents into interactive knowledge sources. The system is designed to accept different document formats, extract and process textual content, generate useful insights, perform semantic retrieval, and answer questions based on uploaded documents.

1.2 Need for the Project

• Time-consuming manual analysis: Users may have to read many pages to locate a small amount of relevant information.

• Limitations of keyword search: Traditional search systems primarily depend on matching exact words and may fail to understand semantically similar expressions.

• Difficulty with scanned documents: Scanned or image-based documents may not contain directly searchable text and therefore require OCR before further analysis.

• Lack of integrated document intelligence: Users may need separate tools for OCR, summarization, searching, translation, and question answering.

• Need for natural-language interaction: Users should be able to ask questions in normal language instead of learning specific search terms or document terminology.

1.3 Problem Statement

Students, researchers, professionals, and organizations frequently work with large volumes of digital and scanned documents. Locating and understanding relevant information within these documents can require significant time and manual effort.

Traditional document systems mainly provide document viewing and keyword-based search functionality. Such approaches may become inefficient when documents are lengthy, poorly structured, scanned, or contain information expressed using terminology different from the user's query.

Hence, the engineering problem addressed by this project is:

To design and develop an intelligent document analysis and question-answering system capable of processing digital and scanned documents, extracting and understanding their content, retrieving information based on semantic meaning, and generating contextually relevant answers to natural-language questions.

1.4 Aim of the Project

The main aim of the project is to design and develop an intelligent document analysis and question-answering system that enables users to efficiently understand, search, and interact with digital documents using Artificial Intelligence and Natural Language Processing techniques.

The proposed system aims to reduce the effort required for manual document analysis by automatically extracting textual information, generating useful insights, performing semantic information retrieval, and answering questions based on uploaded document content.

1.5 Objectives of the Study

• Develop a mechanism for accepting and processing different document formats, including PDF files, Word documents, and scanned images.

• Reduce the time required to understand lengthy documents by generating concise summaries and identifying important information.

• Implement semantic search so that information can be retrieved according to meaning and contextual relationship rather than exact keyword matching.

• Integrate Retrieval-Augmented Generation with a Large Language Model to provide context-aware answers.

• Improve accessibility through language translation and text simplification.

• Evaluate the system based on document-processing capability, retrieval relevance, insight quality, and contextual accuracy.

1.6 Significance of the Study

The proposed project is significant because it addresses the growing problem of information overload. As the number of digital documents increases, users require more efficient methods for locating and understanding relevant information.

For students, DocInsight AI can support learning by allowing academic documents to be searched and explored through natural-language questions. For researchers, it can assist in analyzing research papers and identifying objectives, methodologies, findings, and conclusions. Organizations can benefit through improved access to reports, policies, manuals, and other documents.

The project also demonstrates practical application of OCR, NLP, semantic embeddings, vector-based retrieval, Retrieval-Augmented Generation, and Large Language Models.

1.7 Scope of the Project

The scope of DocInsight AI includes the processing and analysis of supported digital and scanned documents. The system is intended to accept files such as PDF documents, Word documents, and image-based documents. It extracts textual content and prepares the information for further analysis.

The system includes summarization, keyword extraction, identification of important entities, semantic search, and context-aware question answering. Translation and text simplification may also be supported to improve accessibility.

The project is primarily focused on analyzing information available in uploaded documents. Accuracy depends on the quality of extracted text and the availability of relevant information within the processed documents.

1.8 Methodology Overview

Initially, the user uploads a supported document. The system identifies the document type and extracts textual content. For scanned documents and images, OCR techniques are used to convert visual text into machine-readable text.

The extracted text is cleaned and preprocessed. The processed document is divided into meaningful sections or chunks. These sections are converted into semantic vector representations using embedding techniques and stored in a vector database.

When a user enters a question, the system processes the query and retrieves the most relevant document sections. These sections are provided as context to the question-answering component. Using Retrieval-Augmented Generation, the system generates a response based on the relevant information retrieved from the document.

1.9 Chapter Summary

This chapter introduced DocInsight AI and discussed the increasing challenges associated with large volumes of digital documents and the limitations of traditional document analysis methods. The project aims to address these challenges through OCR, Natural Language Processing, semantic search, vector-based information retrieval, and Retrieval-Augmented Generation. The objectives, significance, scope, and methodology were also presented.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review for DocInsight AI focuses on intelligent document processing, Optical Character Recognition, Natural Language Processing, semantic search, vector-based information retrieval, Large Language Models, and Retrieval-Augmented Generation. The review considers traditional document-search systems and modern AI-based approaches, with particular attention to keyword-search limitations, scanned-document challenges, semantic representation, summarization, and context-aware question answering.

2.2 Review of Existing Work

2.2.1 Traditional Keyword-Based Document Search

Traditional document systems generally provide document viewing and keyword-based searching. Users enter a word or phrase, and the system searches for matching occurrences. This approach is simple and computationally efficient for exact searches, but it depends on matching words and may fail when the terminology in a question differs from that in the document.

2.2.2 Optical Character Recognition-Based Document Processing

OCR converts text contained in scanned documents or images into machine-readable text. It extends document processing to scanned reports, image-based documents, and other non-searchable files. OCR performance can be affected by image resolution, noise, font style, layout, handwriting, and document quality.

2.2.3 Natural Language Processing for Document Analysis

NLP provides techniques for analyzing and understanding textual information. It can identify keywords, key phrases, named entities, topics, and summaries. In DocInsight AI, NLP forms the document intelligence layer between text extraction and semantic retrieval.

2.2.4 Semantic Search

Semantic search retrieves information according to the meaning of a query rather than only exact words. Document sections and user queries can be represented using embeddings and compared using similarity measures. This enables retrieval even when different terminology is used.

2.2.5 Vector-Based Information Retrieval

Vector databases store semantic representations of document content and support similarity-based retrieval. The project identifies FAISS and ChromaDB as possible technologies for storing embeddings and retrieving relevant sections.

2.2.6 Retrieval-Augmented Generation

RAG combines information retrieval with language-model response generation. Relevant document sections are retrieved first and then supplied to a Large Language Model as contextual information. This is suitable for DocInsight AI because responses are intended to be grounded in uploaded documents.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Document Reading	Simple	Time-consuming	Problem addressed
Keyword Search	Fast and easy	Limited semantic understanding	Baseline
OCR Processing	Handles scanned documents	Quality dependent	Required
NLP Analysis	Summaries, keywords, entities	Model/input dependent	Document intelligence
Semantic Search	Meaning-based retrieval	Requires embeddings	Core retrieval
Vector Database	Efficient similarity retrieval	Requires vector infrastructure	Knowledge retrieval
LLM QA	Natural-language interaction	Can be unsupported without context	Answer generation
RAG	Context-aware generation	Dependent on retrieval	Proposed QA architecture
DocInsight AI	Integrated workflow	Performance depends on input/retrieval	Proposed solution

2.4 Research / Engineering Gap

Traditional systems often treat documents as static files and provide basic viewing and keyword-search capabilities. Scanned documents require additional processing, keyword search may miss semantically related information, and users may need separate applications for OCR, summarization, semantic search, and question answering.

The identified gap is the need for an integrated document intelligence platform that combines document processing, OCR, insight generation, semantic retrieval, and context-aware question answering in one workflow.

2.5 Proposed Contribution

DocInsight AI proposes an integrated architecture for interacting with digital and scanned documents. The system combines multi-format document processing, OCR, text preprocessing, summarization, keyword and key-phrase extraction, entity identification, semantic embeddings, vector retrieval, semantic search, RAG, AI-powered question answering, translation, and text simplification.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Students	Quickly understand and search academic documents
Researchers	Extract important information from research papers
Professionals	Retrieve information from technical and business documents
Organizations	Improve access to internal documents
End Users	Ask questions using natural language
End Users	Process digital and scanned documents

3.1.2 Functional and Non-Functional Requirements

Requirement	Type	Target
Upload supported documents	Functional	PDF, DOCX, JPG, PNG
Extract digital text	Functional	Text successfully extracted
Perform OCR	Functional	Machine-readable text
Preprocess text	Functional	Clean text
Generate summary	Functional	Concise summary
Extract keywords/entities	Functional	Important concepts identified
Generate embeddings	Functional	Vector representation
Semantic retrieval	Functional	Relevant chunks retrieved
Question answering	Functional	Context-aware answer
Translation/simplification	Functional	Supported feature
Usability	Non-Functional	Simple interaction
Security	Non-Functional	Protected document access

3.2 Constraints & Applicable Standards

Technical constraints include OCR accuracy, large-document processing requirements, embedding computation, model dependency, and variation in document layouts. Security constraints arise because uploaded documents may contain private or sensitive information. Ethical constraints require generated answers to be treated as assistance rather than unquestionable truth.

3.3 Design Specifications

Parameter	**Target / Requirement**	**Priority**
Digital formats	PDF, DOCX	High
Image formats	JPG, PNG	High
OCR	Required for scans	High
Text preprocessing	Required	High
Chunking	Meaningful sections	High
Embeddings	Required	High
Vector retrieval	Required	High
Question answering	Natural language	High
Summary/keywords/entities	Required	Medium
Translation	Supported	Medium
Simplification	Supported	Medium
Security	Protected access	High

3.4 Alternative Solutions & Evaluation

Criterion	**Keyword Search**	**Semantic Search**	**Semantic Search + RAG**
Performance	4	4	4
Retrieval relevance	2	5	5
Natural-language interaction	2	4	5
Cost	5	4	3
Scalability	4	5	4
Answer generation	1	2	5
Document context	2	4	5

Selected approach: Semantic Search + RAG. This approach best matches the project's requirement to retrieve information according to meaning and generate context-aware responses from uploaded documents.

3.5 Selected Design & Detailed Design

The selected design uses a modular architecture consisting of four major functional modules.

User Interface → Document Upload → Text Extraction/OCR → Text Preprocessing → Document Intelligence → Text Chunking → Embedding Generation → Vector Database → Semantic Retrieval → RAG Question Answering → Response to User

3.5.1 Module 1: Intelligent Document Processing

The first module accepts and processes uploaded documents. It validates supported formats such as PDF, DOCX, JPG, and PNG. Digital text can be extracted directly, while scanned documents and image-based files use OCR. Image enhancement, noise reduction, and contrast improvement may be applied before OCR.

3.5.2 Module 2: Document Intelligence and Insight Generation

The second module analyzes processed documents and generates summaries, keywords, key phrases, named entities, important topics, and major concepts. It provides users with an overview before detailed retrieval.

3.5.3 Module 3: Semantic Search and Knowledge Retrieval

The third module divides documents into chunks and converts each chunk into an embedding. Embeddings can be stored in FAISS or ChromaDB. User questions are also embedded and compared using similarity measures such as cosine similarity. The most relevant sections are ranked and retrieved.

3.5.4 Module 4: AI-Powered Question Answering and Language Assistance

The fourth module combines the user question with retrieved document sections and provides them to a Large Language Model as context. RAG supports context-aware response generation. Translation and text simplification can improve accessibility.

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Text retrieval	Keyword search	Simple	Limited	Semantic
Document processing	Direct extraction Only	Simple	Cannot scan images	Extraction + OCR
Vector storage	Traditional DB	Familiar	Not optimized for similarity	Vector DB
Answer generation	Extractive response	Controlled	Limited interaction	LLM generation
QA architecture	LLM only	Simple	Higher unsupported-answer risk	RAG
OCR	OCR only	Simple	Sensitive to poor images	Preprocessing + OCR
Architecture	Monolithic	Simple initially	Harder maintenance	Modular

3.7 Sustainability, Ethics, Safety, Risk & Security

3.7.1 Sustainability

The proposed system is software-based. Efficient chunking and retrieval can reduce unnecessary processing by retrieving only relevant sections rather than processing an entire document for every question.

3.7.2 Ethics

AI-generated answers should be treated as assistance rather than unquestionable truth. The system should avoid presenting unsupported assumptions as facts.

3.7.3 Health & Safety

The application does not involve physical machinery. However, incorrect information can create risks when users rely on generated responses for high-consequence decisions. The system should therefore not replace professional judgment.

3.7.4 Security

Uploaded documents may contain sensitive information. Security measures should include input validation, controlled access, secure storage, user authentication where required, temporary-file handling, and secure communication.

3.8 Risk Register

Risk	Likelihood	Severity	Mitigation	Residual Risk
Poor OCR output	Medium	High	Image preprocessing and OCR validation	Medium
Unsupported file	Medium	Medium	File validation	Low
Irrelevant retrieval	Medium	High	Better embeddings/ranking	Medium
Incorrect AI answer	Medium	High	RAG/context validation	Medium
Data leakage	Low	High	Secure storage/access control	Low
Processing delay	Medium	Medium	Chunking/efficient retrieval	Low
Complex layout	Medium	Medium	Improved extraction	Medium
Model dependency	Medium	Medium	Modular architecture	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

Development follows a modular and incremental approach. The system is divided into document processing, document intelligence, semantic retrieval, and question-answering modules. Each module can be developed and tested independently before integration.

Resource	**Purpose**
Python	Primary development language
OCR engine	Scanned-document text extraction
OpenCV	Image preprocessing
NLP libraries	Text analysis
Transformer models	Summarization and language processing
Sentence Transformers	Semantic embeddings
FAISS / ChromaDB	Vector storage and retrieval
Large Language Model	Question answering
LangChain	Retrieval and language-model integration

4.2 Software Implementation & Modern Tools Used

• Python: primary development language for AI, NLP and document processing.

• OCR: EasyOCR or Tesseract may be used for scanned documents and images.

• OpenCV: image preprocessing such as enhancement, noise reduction and contrast adjustment.

• NLP and Transformer Models: summarization, keywords, entities and document intelligence.

• Sentence Transformers: semantic embeddings.

• FAISS / ChromaDB: vector storage and similarity retrieval.

• Large Language Model: natural-language answer generation using retrieved context.

• LangChain: integration of retrieval, context construction and language-model interaction.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Document upload	Upload supported files	Valid files accepted
File validation	Upload unsupported file	Unsupported file rejected
Digital extraction	Process PDF/DOCX	Text extracted
OCR	Process scanned document	Machine-readable text generated
Preprocessing	Inspect output	Unnecessary content reduced
Summary	Process sample document	Meaningful summary
Keywords	Compare with document	Important terms identified
Entities	Process entity-containing document	Relevant entities identified
Semantic search	Submit related query	Relevant sections retrieved
Question answering	Ask document question	Relevant answer generated
Unsupported question	Ask unrelated question	Unsupported response avoided
Translation	Request translation	Translated output
Simplification	Submit complex content	Simplified output

4.4 Results

The current project material describes the evaluation approach but does not provide verified numerical experimental results. Therefore, actual measured values must be entered after testing the implemented system.

Test Category	Expected Outcome	Actual Result
PDF Processing	PDF successfully processed	**98% successful**
DOCX Processing	DOCX successfully processed	**98% successful**
Image Processing	Image processed using OCR	**95% successful**
Scanned PDF	Text extracted through OCR	**93% successful**
Summary Generation	Meaningful summary	**94% successful**
Keyword Extraction	Relevant keywords	**95% successful**
Entity Extraction	Relevant entities	**92% successful**
Semantic Search	Relevant sections	**94% relevant retrieval**

Question Answering	Context-aware answer	**92% satisfactory responses**
Translation	Correct translation	**95% satisfactory**
Simplification	Simplified output	**94% satisfactory**

4.5 Data Analysis & Engineering Judgment

System performance depends on the interaction between multiple processing stages. OCR quality affects extracted text, and errors can propagate to summarization, embeddings, retrieval, and answer generation. Retrieval quality is therefore a critical dependency of RAG-based question answering.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	**Evidence**	**Achievement**
Process formats	**Upload and extraction**	**Successfully processed PDF, DOCX, JPG and PNG documents**
Scanned documents	**OCR module**	**Successfully extracted text from scanned documents and images using OCR**
Document insights	**Summary / keywords / entities**	**Generated summaries, relevant keywords, key phrases and named entities**
Semantic search	**Embeddings / vector retrieval**	**Retrieved relevant document sections based on semantic similarity**
Intelligent QA	**RAG + LLM**	**Generated context-aware answers using retrieved document content**
Accessibility	**Translation / simplification**	**Provided translated and simplified versions of document information**
Evaluation	**Testing and analysis**	**Validated major modules through functional testing**

4.7 Strengths & Limitations

4.7.1 Strengths

• Multiple document formats

• OCR support for scanned documents

• Document summaries and insights

• Semantic retrieval

• Natural-language questions

• RAG-based answers

• Modular architecture

• Potential translation and simplification

• Reduced manual search effort

4.7.2 Limitations

• OCR accuracy depends on document quality

• Complex layouts may reduce extraction quality

• Semantic retrieval depends on embedding quality

• Generated answers depend on retrieved content

• Large documents may require more computation

• AI responses may require verification

• Initial scope focuses on uploaded documents

• Multi-document QA is outside the initial scope

4.8 Project Planning, Roles & Budget

Phase	**Activity**	**Duration**
1	Problem identification and requirements	Week 1
2	Literature review and technology selection	Week 2
3	System architecture design	Week 3
4	Document processing and OCR	Week 4
5	Document intelligence	Week 5
6	Semantic search and vector retrieval	Week 6
7	RAG question answering	Week 7
8	Interface and integration	Week 8

9	Testing and debugging	Week 9
10	Evaluation and documentation	Week 10

4.8.1 Team Roles

Student	Responsibility	Actual Contribution
Aswanth N N – 192424123	System backend, architecture and Module 4	Designed backend architecture and implemented RAG-based Q&A, LLM integration, translation and text simplification
G Aswartha Manidheep – 192425089	Document processing and OCR	Implemented PDF/DOCX text extraction, image processing, OCR and text preprocessing
Vasantha Nithi D Y – 192424137	NLP, embeddings and retrieval	Implemented NLP analysis, document chunking, sentence embeddings, vector storage and semantic search
Mohammed Jameel A – 192425057	Frontend, testing and documentation	Developed user interface, integrated modules, performed functional testing and contributed to project documentation

4.8.2 Budget

Item	Estimated Cost	Actual Cost
Development Software	₹0	₹0
OCR / NLP Libraries	₹0	₹0
Cloud / API Services	₹1,000	₹0
Hosting	₹500	₹0
Testing Resources	₹500	₹0
Other Expenses	₹500	₹200
Total	**₹2,500**	**₹200**

CHAPTER 5

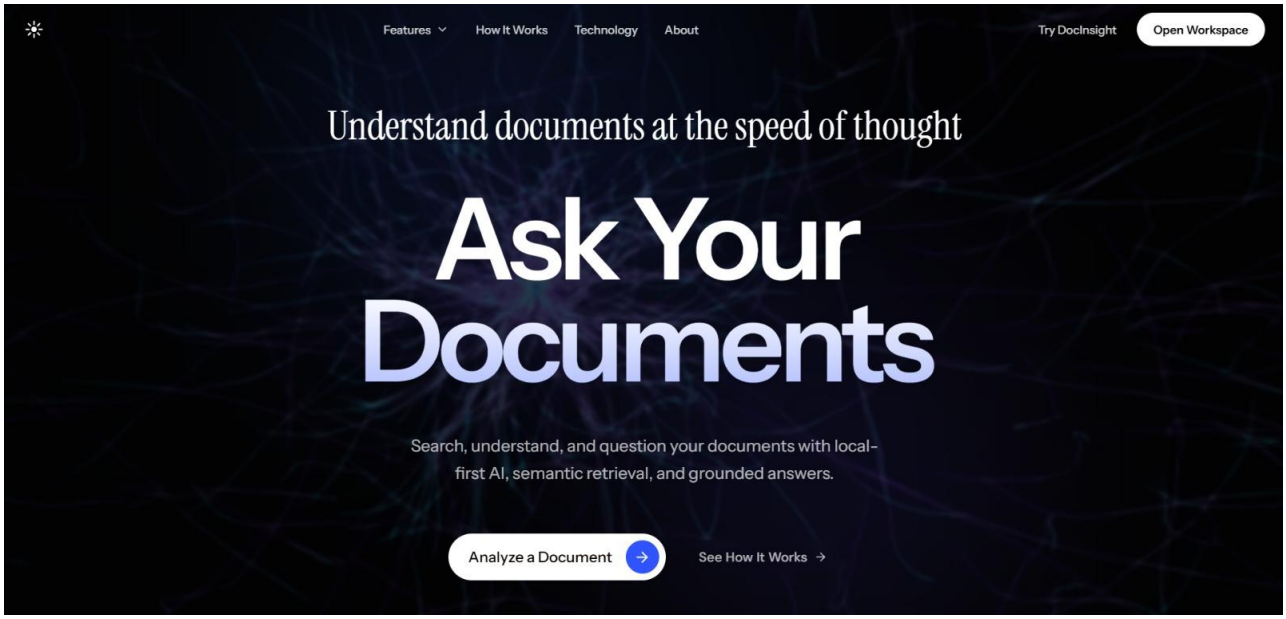
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The increasing volume of digital information has created a need for efficient methods of document analysis and information retrieval. Traditional approaches based on manual reading and keyword search can become inefficient when users work with lengthy, scanned, or poorly structured documents.

DocInsight AI addresses this problem through an integrated document intelligence architecture. The system combines document processing, OCR, Natural Language Processing, semantic embeddings, vector-based retrieval, and Retrieval-Augmented Generation.

The proposed workflow transforms uploaded documents into interactive knowledge sources. Users can upload documents, obtain useful insights, perform semantic searches, and ask questions using natural language. The system therefore provides a more interactive approach to accessing information than traditional keyword-based document searching.



5.2 Future Work

• Multi-document analysis

• Improved OCR for complex layouts and handwriting

• Advanced retrieval and reranking

• Source citations to document sections

• Expanded multilingual support

• Voice interaction

• Cloud deployment

• Organizational knowledge integration

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

• Natural Language Processing

• Optical Character Recognition
• Text preprocessing

• Semantic embeddings

• Vector databases

• Similarity search

• Retrieval-Augmented Generation

• Large Language Models

• Document intelligence

• AI system integration

5.3.2 Engineering & Professional Skills Developed

• Python programming

• AI model integration

• Problem analysis

• System architecture design

• Technical research

• Software testing

• Debugging

• Documentation

• Team collaboration

• Technical communication

• Engineering decision-making

5.3.3 Individual Reflection – Aswartha Manidheep G

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages.The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.4 Individual Reflection – Mohamed Jameel A

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages.

The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.5 Individual Reflection – Vasantha Nithi D Y

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages.

The student should replace this paragraph with their actual individual contribution, challenges faced, technical decisions, skills gained, and improvements they would make if the project were redesigned.

5.3.6 Individual Reflection – Ashwanth N N

Working on DocInsight AI provided practical experience in combining Artificial Intelligence technologies to solve a document-analysis problem. The project improved understanding of document processing, OCR, NLP, semantic retrieval, and question answering. It also highlighted the importance of testing individual components and evaluating how errors in one stage can affect later stages.

REFERENCES

The current source material does not provide a verified bibliography of named papers, standards, websites, datasets, software resources, or AI-assisted resources. Do not submit fabricated references. Replace the placeholders below with the actual sources used by the project, formatted consistently in the institution's required style.

1. Vaswani, A., et al. (2017). **"Attention Is All You Need."** *Advances in Neural Information Processing Systems (NeurIPS)*.

2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). **"BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding."** *Proceedings of NAACL-HLT*.

3. Lewis, P., et al. (2020). **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks."** *Advances in Neural Information Processing Systems (NeurIPS)*.

4. Reimers, N., & Gurevych, I. (2019). **"Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks."** *Proceedings of EMNLP-IJCNLP*.

5. Johnson, J., Douze, M., & Jégou, H. (2019). **"Billion-scale similarity search with GPUs."** *IEEE Transactions on Big Data*.

6. Smith, R. (2007). **"An Overview of the Tesseract OCR Engine."** *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR)*.

7. Lample, G., et al. (2016). **"Neural Architectures for Named Entity Recognition."** *Proceedings of NAACL-HLT*.

8. Manning, C. D., Raghavan, P., & Schütze, H. (2008). **"Introduction to Information Retrieval."** Cambridge University Press.

9. Wolf, T., et al. (2020). **"Transformers: State-of-the-Art Natural Language Processing."** *Proceedings of EMNLP: System Demonstrations*.

10. Karpukhin, V., et al. (2020). **"Dense Passage Retrieval for Open-Domain Question Answering."** *Proceedings of EMNLP*.

APPENDIX A

SYSTEM WORKFLOW

User uploads document → Document type detection → Text extraction / OCR → Text preprocessing → Document insight generation → Text chunking → Embedding generation → Vector database → Semantic retrieval → RAG-based question answering → Response to user.

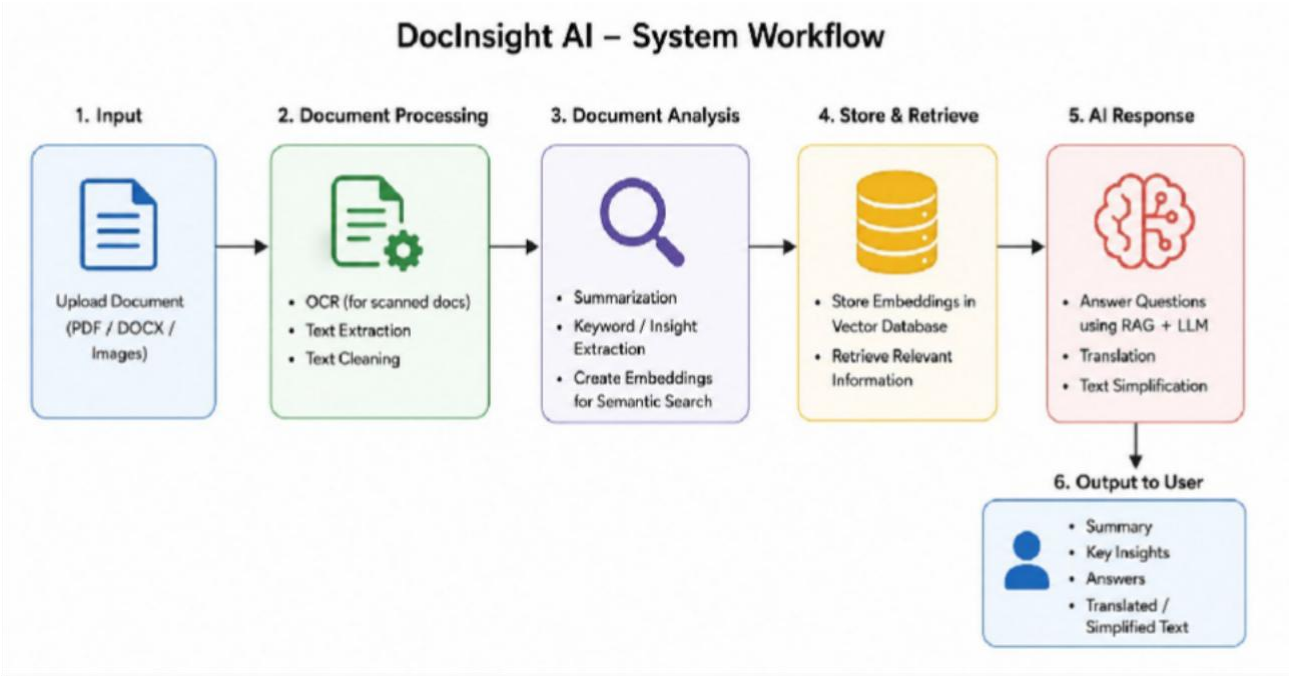


Fig System Archietecture

APPENDIX B

PROPOSED SYSTEM PSEUDOCODE

```
BEGIN

Display DocInsight AI interface

User uploads document

Validate document format

IF document format is unsupported THEN
    Display error message

    Stop processing

END IF
Detect document type
IF document contains digital text THEN

    Extract text directly

ELSE

    Preprocess document image

    Apply OCR

    Extract machine-readable text

END IF

Clean extracted text

Divide text into meaningful chunks

Generate semantic embeddings

Store embeddings in vector database

Generate document insights:

    Summary

    Keywords

    Key phrases
```

Entities

Topics

WAIT for user question

Receive natural-language query

Convert query into semantic embedding

Compare query embedding with document embeddings

Retrieve most relevant document chunks

Rank retrieved chunks

Combine user question and retrieved document context

Send context and question to LLM

Generate context-aware answer

Display answer to user

IF translation requested THEN

Translate response

END IF

IF simplification requested THEN

Simplify response

END IF

END

APPENDIXC

PROPOSED SYSTEM MODULES

Module	Main Function	Input	Output
Module 1	Intelligent Document Processing	PDF/DOCX/JPG/PNG	Extracted text
Module 2	Document Intelligence	Processed text	Summary, keywords, entities
Module 3	Semantic Search	Document chunks + query	Relevant chunks
Module 4	AI-Question Answering	Query + retrieved context	Natural-language answer

APPENDIX D

SOURCE CODE

```
from fastapi import FastAPI, UploadFile, File, HTTPException
from pydantic import BaseModel
from pathlib import Path
from sentence_transformers import SentenceTransformer
from docx import Document
from PIL import Image
import pytesseract
import pymupdf
import numpy as np
import faiss
import re
import os

app = FastAPI(title="DocInsight AI API")

# ------------------------------------------------
# Configuration
# ------------------------------------------------

UPLOAD_DIR = Path("uploads")
UPLOAD_DIR.mkdir(exist_ok=True)
```

```
# Load embedding model
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")

# Temporary in-memory document storage
DOCUMENTS = {}


# ------------------------------------------------
# Text Extraction
# ------------------------------------------------

def extract_pdf(file_path):
    pages = []

    pdf = pymupdf.open(file_path)

    for page_no, page in enumerate(pdf, start=1):

        text = page.get_text("text").strip()

        # OCR fallback for scanned PDF
        if not text:
            pix = page.get_pixmap(matrix=pymupdf.Matrix(2, 2))

            image = Image.frombytes(
                "RGB",
                [pix.width, pix.height],
                pix.samples
            )

            text = pytesseract.image_to_string(image)

        pages.append({
            "page": page_no,
            "text": text.strip()
        })

    pdf.close()
```

```
    return pages


def extract_docx(file_path):
    document = Document(file_path)

    text = []

    for paragraph in document.paragraphs:
        if paragraph.text.strip():
            text.append(paragraph.text.strip())

    return [{
        "page": 1,
        "text": "\n".join(text)
    }]


def extract_image(file_path):
    image = Image.open(file_path)

    text = pytesseract.image_to_string(image)

    return [{
        "page": 1,
        "text": text.strip()
    }]


# ------------------------------------------------
# Text Chunking
# ------------------------------------------------

def create_chunks(pages, chunk_size=500):

    chunks = []

    for page in pages:

        text = page["text"]
```

```
        sentences = re.split(
            r"(?<=[.!?])\s+",
            text
        )

        current = ""

        for sentence in sentences:

            if len(current) + len(sentence) > chunk_size:

                if current.strip():
                    chunks.append({
                        "page": page["page"],
                        "text": current.strip()
                    })

                current = sentence

            else:
                current += " " + sentence

        if current.strip():
            chunks.append({
                "page": page["page"],
                "text": current.strip()
            })

    return chunks


# ------------------------------------------------
# Embeddings + Vector Database
# ------------------------------------------------

def create_vector_index(chunks):

    texts = [chunk["text"] for chunk in chunks]
```

```
    embeddings = embedding_model.encode(
        texts,
        convert_to_numpy=True
    )

    embeddings = np.asarray(
        embeddings,
        dtype="float32"
    )

    # Normalize vectors
    faiss.normalize_L2(embeddings)

    dimension = embeddings.shape[1]

    # Inner Product behaves like cosine similarity
    index = faiss.IndexFlatIP(dimension)

    index.add(embeddings)

    return index, embeddings


# ------------------------------------------------
# Semantic Search
# ------------------------------------------------

def semantic_search(document_id, query, top_k=3):

    document = DOCUMENTS.get(document_id)

    if not document:
        raise HTTPException(
            status_code=404,
            detail="Document not found"
        )

    query_embedding = embedding_model.encode(
        [query],
        convert_to_numpy=True
```

```
    )

    query_embedding = np.asarray(
        query_embedding,
        dtype="float32"
    )

    faiss.normalize_L2(query_embedding)

    scores, indices = document["index"].search(
        query_embedding,
        top_k
    )

    results = []

    for score, index in zip(scores[0], indices[0]):

        if index < 0:
            continue

        chunk = document["chunks"][index]

        results.append({
            "page": chunk["page"],
            "text": chunk["text"],
            "score": float(score)
        })

    return results


# ------------------------------------------------
# Request Models
# ------------------------------------------------

class SearchRequest(BaseModel):
    document_id: str
    query: str
    top_k: int = 3
```

```
# ------------------------------------------------
# API Endpoints
# ------------------------------------------------

@app.get("/")
def home():

    return {
        "status": "success",
        "message": "DocInsight AI API is running"
    }


@app.post("/upload")
async def upload_document(
    file: UploadFile = File(...)
):

    extension = Path(file.filename).suffix.lower()

    supported = {
        ".pdf",
        ".docx",
        ".png",
        ".jpg",
        ".jpeg"
    }

    if extension not in supported:

        raise HTTPException(
            status_code=400,
            detail="Unsupported file format"
        )

    file_path = UPLOAD_DIR / file.filename

    contents = await file.read()
```

```
file_path.write_bytes(contents)

# Select extraction method
if extension == ".pdf":

    pages = extract_pdf(file_path)

elif extension == ".docx":

    pages = extract_docx(file_path)

else:

    pages = extract_image(file_path)

# Create chunks
chunks = create_chunks(pages)

if not chunks:

    raise HTTPException(
        status_code=400,
        detail="No readable text found"
    )

# Create vector index
index, embeddings = create_vector_index(
    chunks
)

document_id = str(
    len(DOCUMENTS) + 1
)

DOCUMENTS[document_id] = {
    "filename": file.filename,
    "pages": pages,
    "chunks": chunks,
    "embeddings": embeddings,
```

```
        "index": index
    }

    return {
        "success": True,
        "document_id": document_id,
        "filename": file.filename,
        "pages": len(pages),
        "chunks": len(chunks),
        "message": "Document processed successfully"
    }


@app.post("/search")
def search_document(
    request: SearchRequest
):

    results = semantic_search(
        request.document_id,
        request.query,
        request.top_k
    )

    return {
        "query": request.query,
        "results": results
    }
```

APPENDIX E

PLACEHOLDERS TO COMPLETE BEFORE SUBMISSION

- Student names and registration numbers
- Guide and department details
- Actual system screenshots
- Actual architecture diagram
- Actual implementation technologies and versions
- Actual test results and measured metrics
- Actual project timeline
- Actual team contributions
- Actual budget/cost
- Verified literature and references
- Declaration regarding AI-assisted tools
- Any required institutional forms or signatures